



UNIVERSITÀ DEGLI STUDI DI SALERNO



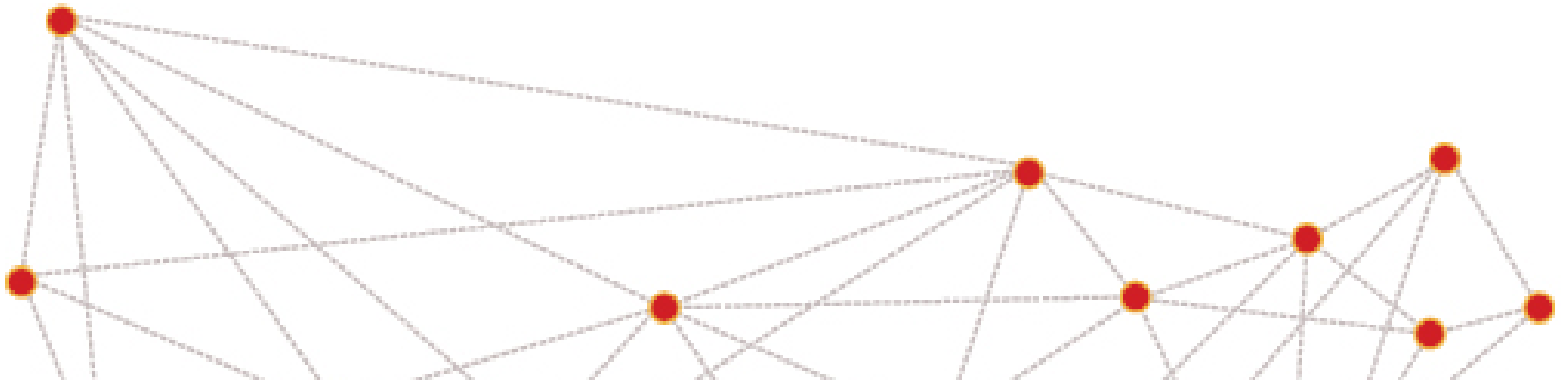
UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA
DIPARTIMENTO DI ECCELLENZA



Prof. Carmen Bisogni

C.A.S.A. Course

CONTEXT AWARE SECURITY ANALYTICS IN COMPUTER VISION





Transformer for Computer Vision: Introduction

Transformers have revolutionized natural language processing (NLP) tasks by enabling models like BERT and GPT to achieve state-of-the-art performance.

Recently, the Transformer architecture has been adapted for computer vision tasks, leveraging its strengths in capturing long-range dependencies and global context understanding.

Vision Transformer (ViT) have recently emerged as a competitive alternative to Convolutional Neural Networks (CNNs) that are currently state-of-the-art in different image recognition computer vision tasks. ViT models outperform the current state-of-the-art (CNN) by almost x4 in terms of computational efficiency and accuracy.



Transformer for Computer Vision: Introduction

The standard Transformer architecture is composed of two main components:

- **Encoder:** Processes the input sequence.
- **Decoder:** Generates the output sequence (used in sequence-to-sequence models).

Each encoder/decoder block contains:

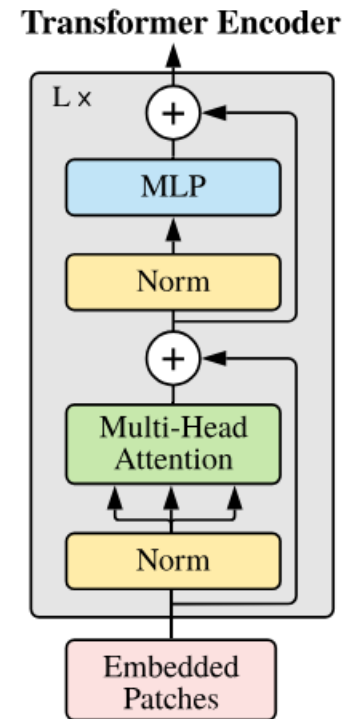
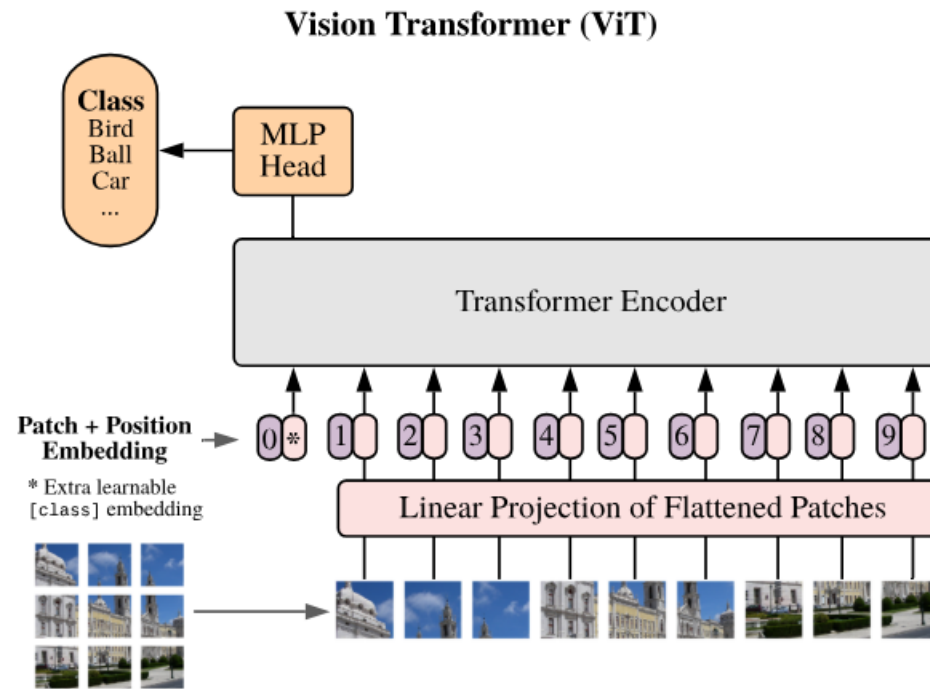
- **Multi-Head Self-Attention**
- **Feed-Forward Neural Network (MLP)**
- **Add & Layer Normalization**
- **Residual Connections**

In vision tasks (e.g., Vision Transformer), only the encoder is typically used.



Transformer for Computer Vision: Introduction

The Vision Transformer (ViT) model architecture was introduced in a research paper published as a conference paper at ICLR 2021 titled “An Image is Worth 16*16 Words: Transformers for Image Recognition at Scale”. It was developed and published by Neil Houlsby, Alexey Dosovitskiy, and 10 more authors of the Google Research Brain Team.



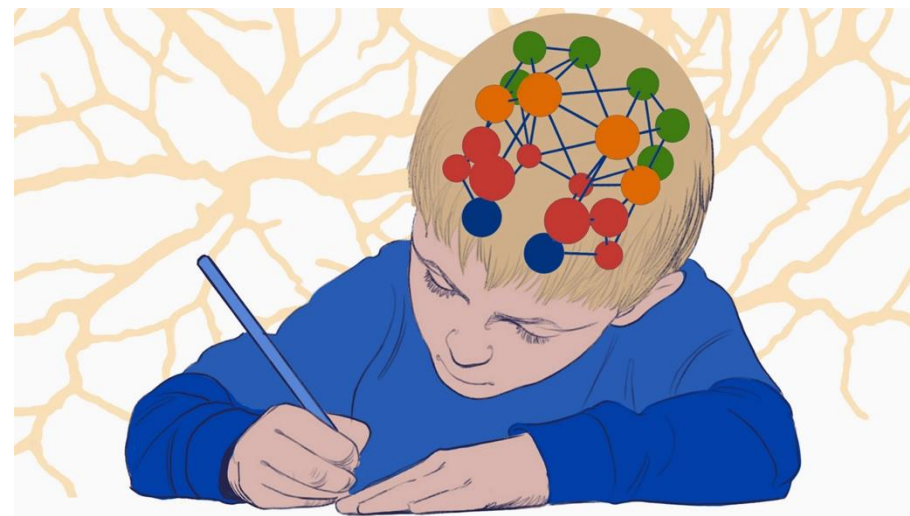


Transformer for Computer Vision: Structure



A transformer in machine learning is a deep learning model that uses the mechanisms of attention, differentially weighing the significance of each part of the input sequence of data. Transformers in machine learning are composed of multiple self-attention layers.

What is an attention (and self-attention) layer?

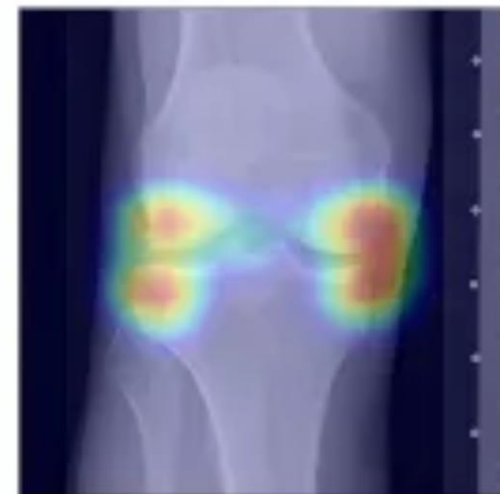




Transformer for Computer Vision: Attention Module

Attention is a concept borrowed from human cognition, where we selectively pay attention to certain aspects of our sensory input and ignore others. For example, when we look at a picture, we can focus on a specific object or region and filter out the background noise.

In computer vision, attention mechanisms are methods that mimic this behaviour by assigning different weights or scores to different parts of the input, such as pixels, regions, or features. These weights or scores indicate how much the model should pay attention to each part, and they are used to modify the output of the model or the input of the next layer.



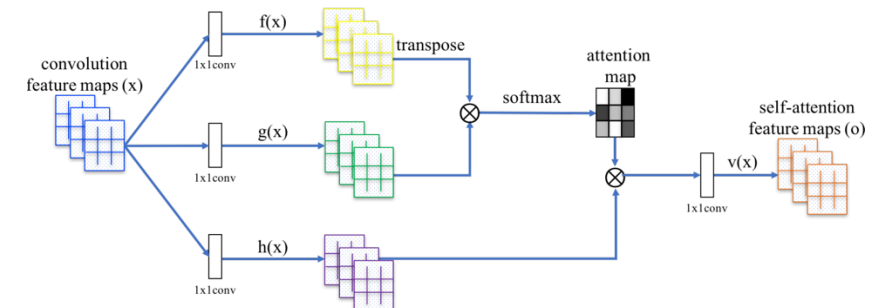
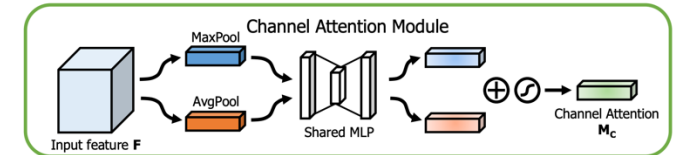
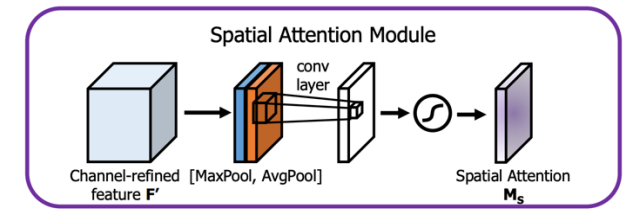


Transformer for Computer Vision: Attention Module



Different types of attention mechanisms can be used in computer vision tasks, depending on the structure and goal of the model:

- Spatial attention
- Channel attention
- Self-attention

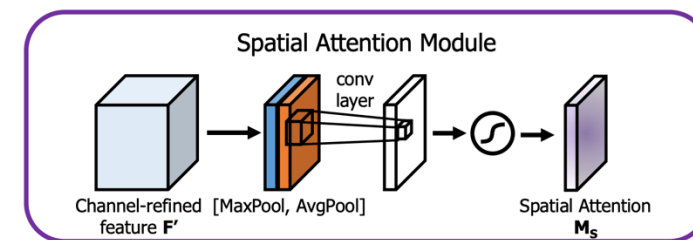




Transformer for Computer Vision: Attention Module



- Spatial attention: Assigns different weights to each spatial location in the input image—i.e., every pixel or region (height $\times$ width). Helps the model focus on where to look in the image.
The model computes a score for each spatial location. Important regions receive higher weights. These weights modulate the feature maps to emphasize key areas.



Example: In an image of a cat on a couch, spatial attention may assign higher weights to the cat's body or face, and lower weights to the background.

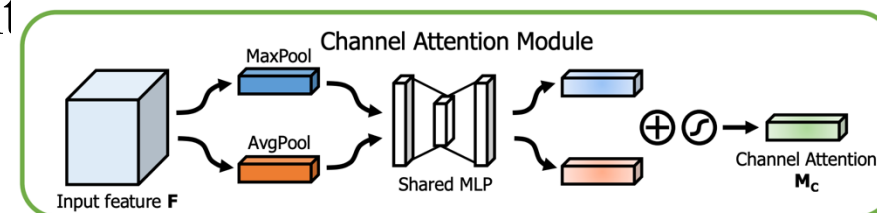


Transformer for Computer Vision: Attention Module



- Channel Attention: Assigns weights to each channel (feature map) of a convolutional layer. Helps the model focus on what features are most important (edges, colors, textures, etc.).

Each channel is scored based on its importance. The network amplifies useful channels and suppresses less relevant



Example: In a medical image, the channel that highlights fine lines or fractures might get a higher weight when looking for bone fractures.



Transformer for Computer Vision: Attention Module

- Self Attention: Computes the relationships between all parts of the input, regardless of spatial distance. Captures long-range dependencies and interactions between different image regions. Each pixel, patch, or token computes attention scores with every other part of the image. This creates an attention map showing how strongly each element relates to others.

Example: In an image of a zebra, self-attention may relate the stripes on the front and back of the animal, even if they are far apart in the image, helping the model understand they belong to the same object.



Transformer for Computer Vision: Attention Module

When selecting the best type of attention for computer vision tasks, there is no definitive answer, as it depends on the nature and goal of the task, the structure and design of the model, and the availability and quality of the data. For example, spatial attention may be more suitable for tasks that involve locating or segmenting objects, while channel attention may be more suitable for tasks that involve extracting or classifying features. Self-attention may be more suitable for tasks that involve understanding or generating complex or diverse inputs.

Transformers may benefit from self-attention to capture their long-range and global processing...



Transformer for Computer Vision: from CNN to ViT

The ViT model represents an input image as a series of image patches, like the series of word embeddings used when using transformers to text, and directly predicts class labels for the image.

CNN uses pixel arrays, whereas ViT splits the input images into visual tokens. The visual transformer divides an image into fixed-size patches, correctly embeds each of them, and includes positional embedding as an input to the transformer encoder.

The self-attention layer in ViT makes it possible to embed information globally across the overall image. The model also learns from training data to encode the relative location of the image patches to reconstruct the structure of the image.



Transformer for Computer Vision: attention maps of ViT

The attention maps of Vision Transformer (ViT) are matrices that represent the importance of different parts of an input image to different parts of the model's learned representations. In ViT, the entire image of the input data is first divided into non-overlapping patches, which are then flattened and fed into the transformer encoder.

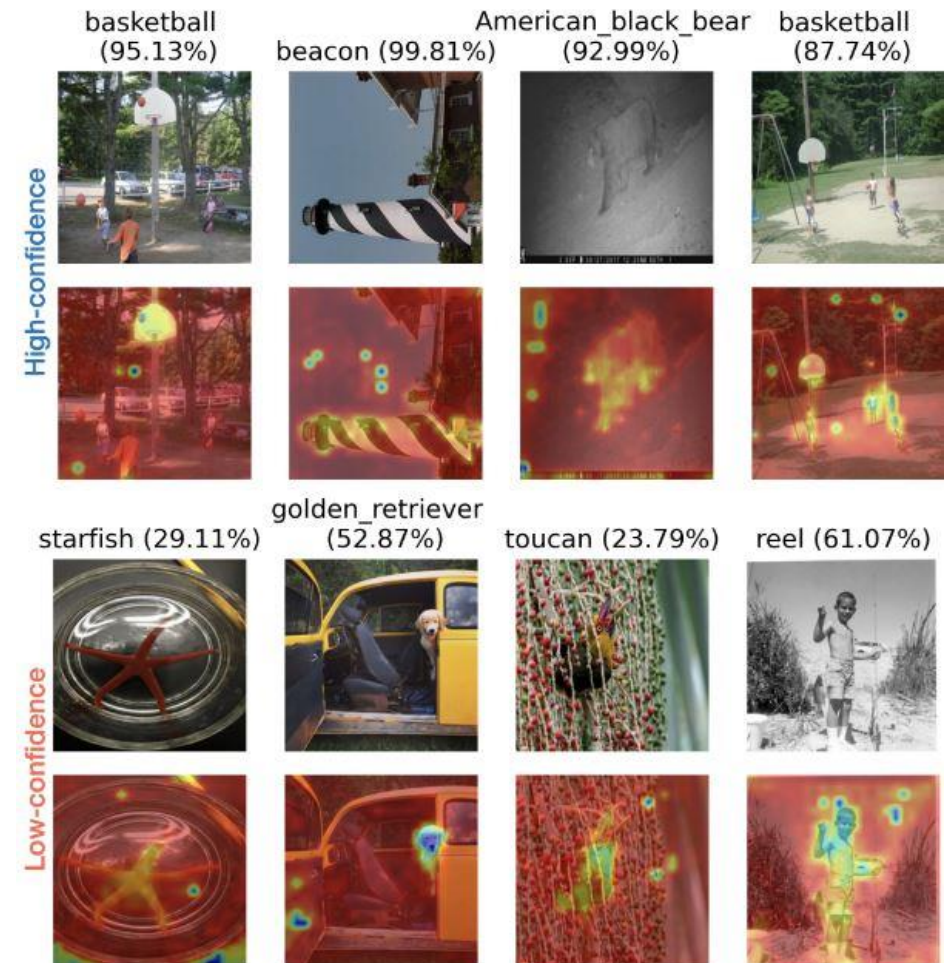
Attention maps refer to the visualizations of the attention weights that are calculated between each token (or patch) in the image and all other tokens. These attention maps are calculated using a self-attention mechanism, where each token attends to all other tokens to obtain a weighted sum of their representations.



Transformer for Computer Vision: attention maps of ViT – Part 2



The attention maps can be visualized as a grid of heatmaps, where each heatmap represents the attention weights between a given token and all other tokens. The brighter the color of a pixel in the heatmap, the higher the attention weight between the corresponding tokens. By analyzing the attention maps, we can gain insights into which parts of the image are most important for the classification task at hand.





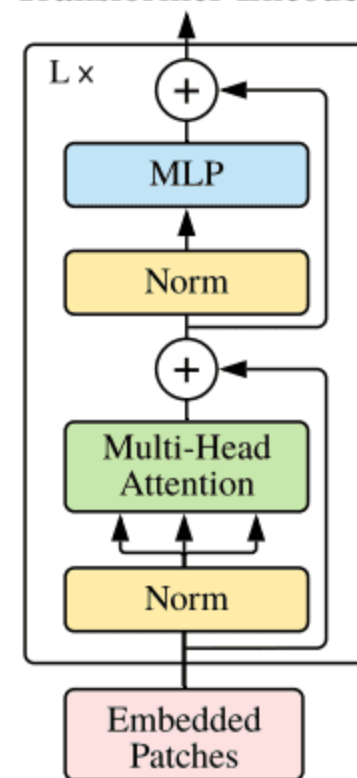
Transformer for Computer Vision: architecture



The transformer encoder includes the following:

- **Multi-Head Self Attention Layer (MSP):** This layer concatenates all the attention outputs linearly to the right dimensions. The many attention heads help train local and global dependencies in an image.
- **Multi-Layer Perceptrons (MLP) Layer:** This layer contains a two-layer with Gaussian Error Linear Unit (GELU) as activation function.
- **Layer Norm (LN):** This is added prior to each block as it does not include any new dependencies between the training images. This thereby helps improve the training time and overall performance.

Transformer Encoder



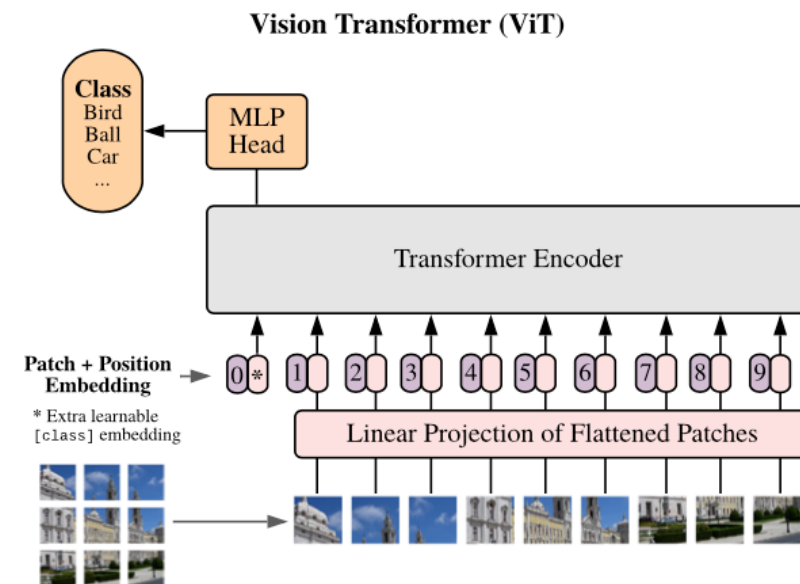


Transformer for Computer Vision: architecture



The overall architecture is:

1. Split an image into patches (fixed sizes).
2. Flatten the image patches.
3. Create lower-dimensional linear embeddings from these flattened image patches (dense layer).
4. Include positional embeddings (say where the patch is).
5. Feed the sequence as an input to a state-of-the-art transformer encoder.
6. Pre-train the ViT model with image labels, which is then fully supervised on a big dataset.
7. Fine-tune the downstream dataset for image classification.





Transformer for Computer Vision: optimization

Compared to ViT, CNNs are easier to optimize.

For this reason, the idea is to compose a transformer with a CNN front-end.

The usual ViT stem leverages a 16×16 convolution with a 16 stride. In comparison, a 3×3 convolution with stride 2 increases the stability and elevates precision.

CNN turns basic pixels into a feature map. Later, the feature map is translated by a tokenizer into a sequence of tokens that are then inputted into the transformer. The transformer then applies the attention technique to create a sequence of output tokens. Eventually, a projector reconnects the output tokens to the feature map. The latter allows the examination to navigate potentially crucial pixel-level details. This thereby lowers the number of tokens that need to be studied, lowering costs significantly.



Transformer for Computer Vision: CNN or ViT?

If the ViT model is trained on huge datasets that are over 14M images, it can outperform the CNNs. If not, the best option is to stick to ResNet or EfficientNet.

CNN vs ViT code at: <https://medium.com/@faheemrustamy/vision-transformers-vs-convolutional-neural-networks-5fe8f9e18efc>



Transformer for Computer Vision: challenges

- Transformers lack some inductive biases compared to CNNs, and rely heavily on massive datasets for large-scale training, which is why the quality of data significantly influences the generalization and robustness of transformers in computer vision tasks.
- Whilst ViT shows exceptional performance on downstream image classification tasks, for example, VTAB and CIFAR, directly applying the ViT backbone on object detection has failed to surpass the results of CNNs
- It still remains a challenge to fully understand why transformers work well on visual tasks.
- It is difficult to develop efficient transformer models for computer vision that can be deployed on resource-limited devices.



Evaluation metrics in ML and DL: A summary overview

- Accuracy: the simplest and most straightforward metric for classification tasks. It's the ratio of correct predictions to total predictions. While accuracy can be useful, it's important to note that it can be misleading if your classes are imbalanced.

TP=True Positive

TN=True Negative

FP=False Positive

FN=False Negative

$$\textit{accuracy} = \frac{TP + TN}{TP + FN + TN + FP}$$



Evaluation metrics in ML and DL: A summary overview

- Precision: measures the accuracy of positive predictions in a classification task. It's the ratio of true positives (items correctly labeled as positive) to all positives, i.e., the sum of true positives and false positives (items incorrectly labeled as positive).

$$\textit{precision} = \frac{TP}{TP + FP}$$



Evaluation metrics in ML and DL: A summary overview

- Recall (also known as Sensitivity or True Positive Rate): This metric measures the fraction of positives that were correctly identified in a classification task, i.e., the ratio of true positives to the sum of true positives and false negatives (positives incorrectly labeled as negative).

$$\textit{recall} = \frac{TP}{TP + FN}$$



Evaluation metrics in ML and DL: A summary overview

- **F1-Score:** is the harmonic mean of precision and recall. It tries to find the balance between precision and recall in a classification task. The F1 score is particularly useful when you have uneven class distribution, as it seeks a balance between precision and recall.

$$\mathbf{F1\ Score} = \frac{2 \times \mathbf{precision} \times \mathbf{recall}}{\mathbf{precision} + \mathbf{recall}}$$



Evaluation metrics in ML and DL: A summary overview



- A confusion matrix is a table layout that visualizes the performance of an algorithm. Each row of the matrix represents the instances in an actual class while each column represents the instances in a predicted class, or vice versa. The name stems from the fact that it makes it easy to see if the system is confusing two classes. This matrix is mainly applicable for classification tasks.

		Real Label		
		Positive	Negative	
Predicted Label	Positive	True Positive (TP)	False Positive (FP)	Precision = $\frac{\sum TP}{\sum TP + FP}$
	Negative	False Negative (FN)	True Negative (TN)	

Recall = $\frac{\sum TP}{\sum TP + FN}$

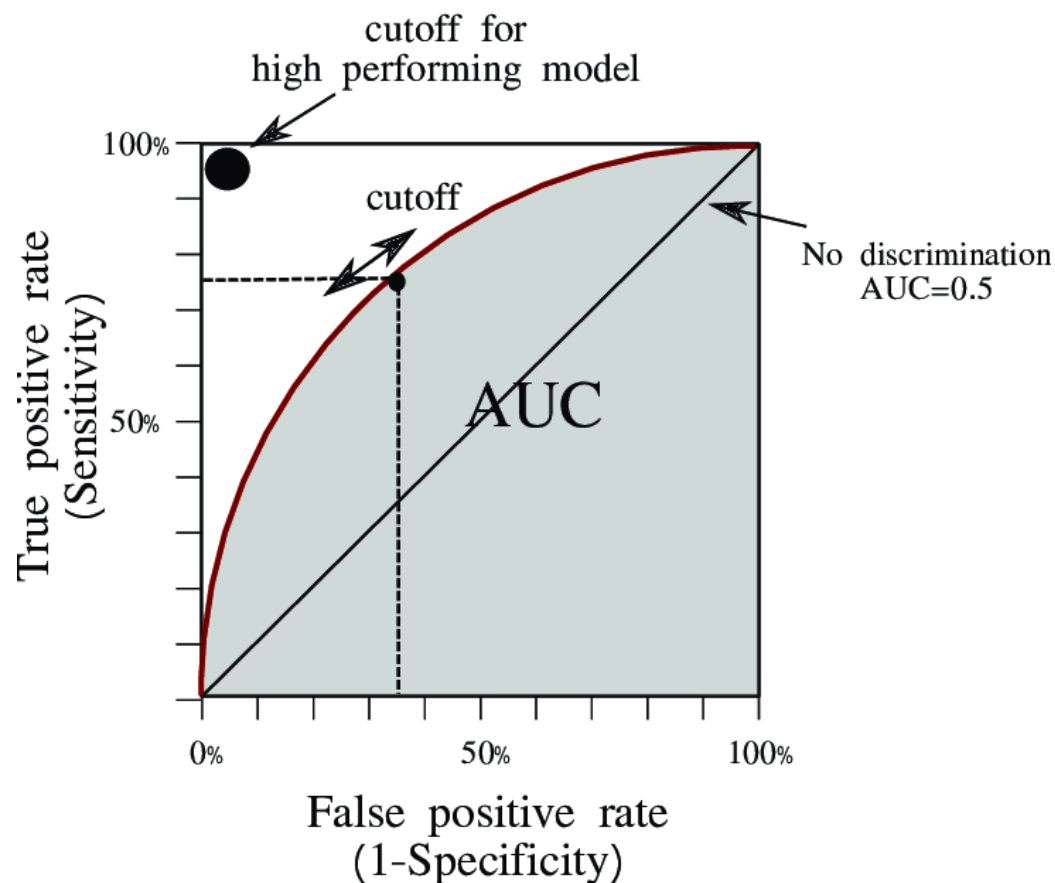
Accuracy = $\frac{\sum TP + TN}{\sum TP + FP + FN + TN}$



Evaluation metrics in ML and DL: A summary overview



- The ROC (Receiver Operating Characteristic) curve is a plot that illustrates the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings. The AUC (Area Under the Curve) measures the entire two-dimensional area underneath the entire ROC curve from (0,0) to (1,1). A model with a perfect predictive performance will have an AUC of 1, whereas a model with random predictive performance will have an AUC of 0.5. This is typically relevant for classification problems.





Evaluation metrics in ML and DL: Examples



In order to evaluate the above mentioned metrics in Python, a classification report should be performed.

```
[122]: # Get predictions
pred = vgg_conv.predict(test_data, batch_size=16)
pred = np.argmax(pred, axis=-1)

# Original labels
labels = np.argmax(test_labels, axis=-1)
from sklearn.metrics import classification_report
print(classification_report(labels, pred))
```

	precision	recall	f1-score	support
0	0.99	0.54	0.70	234
1	0.78	1.00	0.88	390
accuracy			0.83	624
macro avg	0.89	0.77	0.79	624
weighted avg	0.86	0.83	0.81	624

	precision	recall	f1-score	support
0	0.00	0.00	0.00	33
1	0.00	0.00	0.00	24
2	0.00	0.00	0.00	23
3	0.67	0.83	0.74	789
4	0.58	0.67	0.62	575
5	0.50	0.03	0.06	61
6	0.00	0.00	0.00	34
7	0.28	0.04	0.07	122
accuracy			0.63	1661
macro avg	0.25	0.20	0.19	1661
weighted avg	0.56	0.63	0.58	1661



Evaluation metrics in ML and DL: Examples



Do not rely only on accuracy! Expecially if your test dataset is unbalanced!

Classification Report :

	precision	recall	f1-score	support
0	1.00	1.00	1.00	28432
1	0.02	0.02	0.02	49
accuracy			1.00	28481
macro avg	0.51	0.51	0.51	28481
weighted avg	1.00	1.00	1.00	28481

	precision	recall	f1-score	support
0	0.96	0.97	0.97	3391
1	0.62	0.52	0.57	287
accuracy			0.94	3678
macro avg	0.79	0.75	0.77	3678
weighted avg	0.93	0.94	0.94	3678



Evaluation metrics in ML and DL: Examples

How to avoid this situation? Balance of the dataset! How?

- Perform test on the same amount of image per classes, by considering subsets
- Use Data Augmentation to have the same number of images
- Use weighted techniques (for the loss) to balance between under and over represented classes.

Unbalance could also affect Training (in case of wide unbalance).

In this case, apply the above-mentioned techniques also at the training set.